INTERNAL — PRODUCT & ENGINEERING

Guest CRM — Product Requirements Document

The complete product specification: purpose, architecture, every subsystem and how they interlock — plus the when/if/then connection to the Chekin Automation Engine.

Version: 1.0 — July 2026

For: Engineering, Design & Product

Tool: Chekin Platform · Guest CRM module

1 Purpose and problem statement

Turning operational exhaust into a revenue engine

Property managers and hoteliers using Chekin already generate a continuous stream of first-party guest data as a **by-product of operations**: reservations, online check-ins, identity verification, inbox conversations, upsell purchases. Today that data is fragmented across operational silos and monetized only opportunistically — per-stay upsells at check-in time.

The problem: the guest relationship ends at checkout. There is no persistent profile, no memory of preferences or consent, no way to reach the right guest with the right offer at the right moment — and no way to do any of this at scale without a marketing team the typical property manager does not have.

👤

Unified profiles

Every guest becomes a persistent profile assembled automatically — zero data entry.

✨

Vela, the agent

Detects revenue opportunities, drafts audiences, campaigns and copy from a one-sentence brief.

🛡️

Consent-first

A compliance layer makes it structurally impossible to message anyone who has not opted in.

✅

Human-approved

A person approves everything before anything sends. The AI proposes; the operator decides.

> **The one-line pitch:** Your bookings already know who your best guests are. Guest CRM lets you act on it — in one sentence, with consent built in.

2 Product principles

Load-bearing rules — features that violate them do not ship

- ✓ **AI-first, human-approved.** Vela drafts everything (segments, campaigns, copy, deal matching), but nothing reaches a guest without explicit human approval. The approval loop is a feature, not friction — it is what makes an autonomous agent trustworthy in a compliance-sensitive domain.
- ✓ **Consent is structural, not decorative.** Consent checking is not a validation step a developer can forget; it is enforced at the audience-resolution layer. A guest without channel consent cannot be part of an eligible audience. Global unsubscribe overrides everything. Some safeguards are hard-locked and cannot be disabled even by admins.
- ✓ **Explainability everywhere.** Every AI output carries its reasoning: segments show the parsed rules, scores list their contributing signals, campaign analytics come with a "Why?". Operators must never have to trust a black box.
- ✓ **Zero data entry.** Profiles, preferences, intents and consent records derive from operations the property already performs (bookings, check-in, inbox). The CRM never asks the operator to type in guest data.
- ✓ **Revenue-denominated.** Every suggestion, campaign and deal is expressed in expected euros — the operator prioritizes by money, not by marketing abstractions.

Users and personas

Persona	Role	What they need
Property manager (primary)	Runs 5–200 units, no marketing staff	One-sentence campaigns, safe defaults, revenue visibility
Revenue / marketing manager	Larger operators and hotel groups	Segmentation control, campaign analytics, deal catalog
Admin / compliance owner	Legal responsibility for communications	Consent audit trail, suppression list, activation safeguards

All three share one interface; the design collapses gracefully — the property manager can ignore Segments and Consent entirely and still operate safely, because the guardrails are structural.

3 System overview

Information architecture and the core loop

Guest CRM is a module inside the Chekin dashboard, organized as seven tabs plus two cross-cutting surfaces:

INFORMATION ARCHITECTURE

```
Guest CRM
├── Overview      command center: Vela hero, KPIs, review queue
├── Guests        unified profile table + guest drawer
├── Segments      natural-language audience builder
├── Campaigns     lifecycle management → Studio + Detail
├── Deals         monetization catalog (own + third-party)
├── Automation    campaign↔trigger bindings on the Engine
└── Consent       categories, rules, suppression, audit
Cross-cutting:
├── Vela          the agent — hero prompt, ⌘K, embedded flows
└── Guest drawer  full profile panel, opens from anywhere
```

How the sections relate — the core loop

CORE LOOP

```
Guests ──feed──▶ Segments ──target──▶ Campaigns ◀──attach── Deals
                                          │
                                  bind via triggers
                                          ▼
conversions ◀── Channels (email · WhatsApp) ◀── Automation Engine
     │
     └──────────── enrich profiles ──────────▶ back to Guests

Consent gates EVERY arrow · Vela drafts at every stage
```

Guests feed segments; segments plus deals compose campaigns; campaigns bind to triggers in the Automation Engine; sends produce conversions that enrich the guest profile — closing the loop. Consent gates every arrow. Vela authors drafts at every stage.

4 Vela — the Guest CRM agent

One agent, one identity, structured and reviewable output

Vela is **one agent with one identity** across the entire dashboard ("Ask Vela", ⌘K) with specialized capabilities inside Guest CRM. It is not a chatbot bolted onto a UI: every Vela capability produces a **structured, editable, reviewable artifact**.

Run type	Input	Output
segment_builder	Natural-language audience description	Structured rule set + eligibility preview
campaign_planner	One-sentence campaign brief	Full plan: audience, deals, copy, trigger, revenue estimate
deal_matching	An audience (rule set)	Ranked deal recommendations with reasons
compliance_check	A campaign	Consent result: eligible/excluded with per-reason counts
performance_summarizer	Campaign analytics	Headline + recommendations
guest_summary	A guest profile	AI headline + next actions with € estimates
suggestion_engine	Whole dataset	Ranked revenue opportunities

Behavioral contract

- ✓ **Draft-only:** every Vela output lands in draft or needs-review state. Vela cannot approve, schedule or send.
- ✓ **Traceable:** every run is recorded (type, input, output, reasoning summary, author, timestamp) and shown in the Agent Activity feed.
- ✓ **Interruptible:** every generated artifact is editable before approval — rules, copy, deals, triggers.

✓ **Revenue-ranked:** when Vela proposes multiple things, they are ordered by expected revenue.

Entry points

🚀

Overview hero

Prompt bar with example chips. Submits into Campaign Studio with the prompt pre-loaded.

⌘

Ask Vela / ⌘K

Global dashboard assistant; inside Guest CRM it shares the module's context.

🔗

Embedded

Suggestion cards, segment insights, guest drawer, filtered guest lists — anything that identifies an audience becomes a campaign in one click.

5 Overview & Guests

The command center and the unified profile

Overview — three questions in one glance

How is guest revenue doing, what has Vela found for me, and what is waiting on me? Composition: the **Vela hero** (the module's identity: tell the agent what you want), a **4-KPI strip** with trend context (campaign revenue, App Sales revenue, consent coverage, active campaigns), **AI suggestions** ranked by revenue with one-click campaign creation, the **Needs-your-review queue** (campaigns with inline Approve, automations, flagged guests — the operational heart of the human-in-the-loop model), **Guest Segmentation** tiles deep-linking into filtered views plus computed insights with actions, top campaigns/ deals, and the **Agent Activity** timeline. Every element is a door into another subsystem with context attached — the Overview never dead-ends.

Guests — one row per human, assembled automatically

A Guest aggregates identity and locale, **reservations** (from Bookings), **check-in sessions** (from Online Check-in), **conversations with detected intents** (from Inbox — e.g. parking, late_checkout), **purchases** (from App Sales), a per-channel **consent summary**, **preferences with visible provenance**, and a lifecycle stage (lead → prospect → guest → in-house → past → repeat → VIP → dormant).

> **Two tag vocabularies, deliberately separate:** Descriptive tags are machine-derived (family, business, summer) and not directly editable. Segment tags are operator decisions (whale, vip, preferred, do_not_contact…). Exclusion tags require a written note and suppress the guest from ALL campaigns.

Opportunity Score — 0–100, rule-based, always explainable

Signal	Points
Upcoming reservation	+25
Repeat guest / VIP	+15
Previous App Sales purchase	+15
High lifetime booking value (≥ €2,000)	+10

Signal	Points
Email marketing consent	+10
Complete profile (≥ 75%)	+10
WhatsApp consent / completed check-in / in-house	+5 each
Global unsubscribe	–20
Exclusion tag	score = 0, tier excluded

Tiers: high ≥ 70, mid ≥ 40, else low. The score is **computed, never stored** — always derived from live profile state. It surfaces as a table pill, a reasons panel in the drawer, and as a segment-rule primitive (opportunityScore > 70). Weights are product-owned constants — tuning them is a product decision, not an ML pipeline.

6 Segments

Natural-language audiences with structured, previewable rules

SEGMENT BUILDER FLOW

User: Families who stayed last summer and have no future booking

Agent: Parsed into 3 rules: lastStaySeason = summer · children > 0 · nextStayAt is false. You can edit each rule before saving.

Result: Preview: 4 guests matched · 3 eligible after consent · 1 excluded (global unsubscribe). Saved as dynamic segment.

- ✓ Operator describes the audience in plain language (or picks an example).
- ✓ Vela parses it into typed rules rendered as editable chips.
- ✓ **Preview runs through the canonical audience matcher** — the same function the consent check uses, so preview numbers and send-time numbers can never disagree.
- ✓ Saved segments are **dynamic**: membership recomputes continuously as guest data changes. The original sentence is kept alongside the rules as provenance.

Rule primitives (one shared vocabulary)

arrival window · party composition · lifecycle stage · language / country · descriptive tags · segment tags · detected intents · check-in status · past-stay season · revenue threshold · Opportunity Score · consent state

The same schema {field, operator, value} is consumed by segment definitions, campaign audience rules and Automation Engine conditions — see Section 10. Segment types: **dynamic** (rule-based, live), **static** (frozen list), **ai_suggested** (Vela-proposed, badge-labeled until adopted). Segment membership changes can emit events (segment_entered) that automations subscribe to.

7 Campaigns & Studio

The unit of outreach, wrapped in an approval workflow

A campaign is an audience, an offer, content, a trigger and analytics. **Anatomy:** objective (upsell · cross-sell · rebooking · retention · operational · check-in recovery); audience (segment reference or inline rules) with its resolved consent check; one channel (email or WhatsApp — the consent check is channel-specific); attached deals, each with Vela's reason; content with personalization tokens ({{firstName}}, {{propertyName}}, {{checkInDate}}); trigger or one-shot schedule; revenue estimate; and the post-send funnel: sent → delivered → opened → clicked → converted.

```
LIFECYCLE

draft ⟶ needs_review ⟶ approved ⟶ scheduled ⟶ active ⇄ paused ⟶
completed

approve() THROWS if the consent check failed
every approval writes an Approval record (who, when, comment)
createdBy distinguishes 'Vela (AI draft)' from human authors
```

> **Invariant:** Approval of a non-compliant campaign is impossible at the service layer — not merely hidden in the UI. No alternative surface can bypass it.

Campaign Studio — from one sentence to a reviewable plan

Six steps: **Brief** → **Audience** → **Deals** → **Content** → **Automation** → **Review**. The brief invokes the campaign planner, which fills all downstream steps at once and shows a live plan summary. Audience uses the same rule editor as Segments with live eligibility preview; Deals shows Vela's ranked recommendations with reasons; Content is generated copy with tokens, tone and variants; Automation selects a trigger from the engine catalog; Review presents the full plan plus consent check → save draft or submit for review. Every context-carrying entry point (suggestion card, segment insight, guest drawer, filtered list, duplicate) lands here with the brief pre-filled — the Studio is the single convergence point for campaign creation.

8 Deals & Automation

The monetization catalog and the engine bindings

Deals — the offer inventory

Each deal: title, description, provider type — **own** (margin-denominated, e.g. Late Checkout at 90% margin) or **third-party** (commission-denominated, e.g. Airport Transfer at 20% via a partner) — price, category (check-in / check-out / transport / wellness / dining / family / business), tags, performance (conversion rate, average revenue), availability constraints and property scope. Each card also shows **which saved segments it fits** — the inverse view of deal matching: "who could I sell this to?". Purchases write back to guest profiles and emit deal_purchased events; deal performance feeds recommendation ranking and revenue estimates.

Automation — the CRM's window into the platform engine

This page does not implement scheduling or delivery — it shows the CRM's **bindings** into the shared Automation Engine. Each row: campaign, trigger (type + value), status (active / paused / needs review), last & next run, runs & conversions, and **consent blocks and failures as first-class counts**. A trigger catalog documents the twelve platform events inline.

> **Dry-run simulator:** "This trigger would send to N guests, block M on consent, sample recipients: …" — never sends. Testing an automation must be free and safe.

Inherited invariant: an automation only runs while its campaign is approved and active; consent is evaluated per-guest at fire time, not at approval time. A guest who revokes consent between approval and trigger is blocked and counted.

9 Consent & first-run

The compliance backbone and the activation experience

Category	Examples	Rule
Transactional	Booking confirmation, check-in instructions, receipts	Always sends — required to deliver the stay
Operational	Access codes, housekeeping, safety notices	Always sends
Marketing	Own upsells, campaigns, rebooking	Requires explicit per-channel opt-in
Partner marketing	Third-party offers (transfers, spa, dining)	Requires a separate explicit opt-in

- ✓ **Channel rules matrix:** email, WhatsApp, SMS, push, inbox — marketing needs consent per channel; transactional always allowed.
- ✓ **Unsubscribe logic:** global unsubscribe blocks all marketing channels at once; per-channel denial blocks only that channel; every marketing send carries an unsubscribe link.
- ✓ **Suppression list:** auto-derived view over consent state — never manually curated — with reason and remaining reachable channels.
- ✓ **Audit trail:** every grant and revocation logged with guest, channel, category, status, source and a proof reference (opt_in_log#114). A GDPR request is a query, not an investigation.
- ✓ **Activation safeguards:** require-approval (default on, toggleable); auto-exclude unconsented guests (hard-locked); block activation on failed consent check (hard-locked).

> **Structural advantage:** Consent capture happens at the operational touchpoints — booking form and online check-in — the same legally required flows that build the profile also capture the permission to use it. Not begged for in a popup.

First-run and activation

A new account sees a coherent, motivating product — not a dashboard of zeros. The service layer serves an empty dataset (static catalogs survive); every page renders a designed empty state that explains what the section will do and offers the next action. The Overview shows a 4-step **activation checklist**: sync guests from Bookings → set consent safeguards → review the deals catalog → create the first campaign with Vela. Progress persists, steps complete as visited, and the Vela hero stays fully functional throughout — the agent is the product; the checklist is the runway.

10 Data model & services

Entities, relations and the layered architecture

```
ENTITY RELATIONS

GUEST 1—N RESERVATION · CONSENT_RECORD · PREFERENCE
        · CONVERSATION · CHECKIN_SESSION · PURCHASE
SEGMENT 1—N CAMPAIGN (or inline audienceRules)
DEAL N—N CAMPAIGN (with attachment reason) · DEAL 1—N PURCHASE
CAMPAIGN 1—1 AUTOMATION (trigger binding) · 1—N APPROVAL
AGENT_RUN ─▶ may produce CAMPAIGN or SEGMENT
TRIGGER_TYPE 1—N AUTOMATION
```

- ✓ **Guest.consentSummary** is a denormalized view over consent records; the records are the source of truth and carry proof references.
- ✓ **Segment.rules and Campaign.audienceRules share one schema** {field, operator, value, label} — the same schema the engine's IF layer consumes.
- ✓ **Campaign.consentCheck** is a materialized result recomputed on demand; approval reads it live.
- ✓ **AgentRun** = {type, input, output, reasoningSummary, createdBy, createdAt, status} — the uniform audit unit for everything Vela does.
- ✓ **Opportunity Score is computed, never stored** — it can never go stale.

Service architecture

```
LAYERS

CRM_DATA (fixtures today / API tomorrow)
 └─ CRMService   the ONLY consumer of CRM_DATA; all reads/writes;
                 sync today, signatures wrap in Promises unchanged
    └─ CRMAgent  Vela capabilities; reads exclusively via service
       └─ CRM helpers + pages: render only via Service/Agent
```

- ✓ **Single data gateway.** No page or helper touches raw data. Swapping fixtures for a REST/GraphQL backend is a change inside CRMService only.
- ✓ **Canonical audience matcher.** One function resolves rules → guests, reused by segment preview, campaign consent check and send-time resolution. Numbers can never disagree.
- ✓ **Invariants live in the service.** approveCampaign throwing on failed consent, exclusion-tag suppression, automation status rules — enforced below the UI so no surface can bypass them.
- ✓ **The agent is a service client.** Vela has no privileged data access; everything it knows is available to the operator too. This keeps explainability honest.

11 Platform integration — zoom out

Guest CRM as a consumer of the Chekin platform

Guest CRM is deliberately a **consumer of the platform**, not a parallel stack. Each module both feeds the CRM and receives value back:

Platform module	Feeds Guest CRM	Receives back
Bookings / Properties	Reservations, party composition, stay dates, value	Rebooking campaigns → direct, channel-fee-free bookings
Online Check-in	Profile fields, check-in status, consent capture at the legally required moment	Check-in recovery campaigns lift completion
Inbox	Conversations, detected intents, sentiment	Intent-triggered offers reduce inbound workload
Upselling / App Sales	Deal catalog, purchases, conversion benchmarks	Campaigns = new distribution channel; attach rate rises
Automation Engine	Trigger events, scheduling, delivery infrastructure	The CRM is its richest client (Section 12)
Legal / Police / Tax	Compliance culture, identity data quality	Consent audit extends the same posture to marketing
Vela (platform-wide)	Shared agent identity, ⌘K surface	Guest CRM is Vela's deepest capability set; its patterns generalize

Open section — decisions pending with the platform team: (a) whether segment-membership events are emitted by the CRM or computed inside the engine; (b) where the consent classifier lives when other modules start sending guest-facing messages; (c) whether Vela's natural-language rule authoring becomes an engine-level service consumed by all modules. Section 12 states our working position on each.

12 The Automation Engine — when / if / then

One rule language for all guest communications and operations

---

The Automation Engine is Chekin's platform-level rule executor. Its mental model — and the one we keep exposing to users — is three words: **WHEN** something happens in the guest journey, **IF** conditions hold, **THEN** do something. Guest CRM does not build its own scheduler, sender or event bus. **A campaign is a when/if/then rule with rich content attached.** Activating a campaign compiles it into an engine rule; pausing suspends the binding; the Automation tab is the CRM's window into those bindings.

WHEN — the event vocabulary is the guest journey

TRIGGER CATALOG

```
reservation_created → checkin_started → checkin_completed / abandoned
→ days_before_arrival(n) → day_of_arrival → during_stay
→ before_checkout → after_checkout
· deal_purchased · campaign_clicked · segment_entered
```

segment_entered is the bridge that makes dynamic segments event sources: a guest whose data changes (new booking, new intent, score crossing 70) enters a segment and that entry fires automations — segmentation becomes a nervous system, not a query tool. **campaign_clicked / deal_purchased** close the loop: engagement events re-enter the engine, enabling multi-step journeys.

IF — one condition vocabulary for the whole platform

The engine's IF layer consumes **the same rule schema as CRM segments**. One vocabulary, three consumers: segment definitions, campaign audience rules, and automation conditions — including non-marketing ones. This is the single most important integration decision: **do not fork the condition language.** A condition authored for marketing ("has children", "asked about parking") must be reusable verbatim in operations ("notify housekeeping when a family checks in").

> **The consent gate is a structural IF.** Every guest-facing THEN carries an implicit, non-removable condition: consent(channel, category) = granted — injected by the engine, not authored by the user, evaluated at FIRE TIME. Revocations between authoring and firing block the send and increment consentBlocks. Every module that sends through the engine inherits compliance for free. Working position: the consent classifier is engine-level middleware.

THEN — actions beyond messaging

Action	Domain	Example
send_campaign_message	Marketing	Late-checkout offer 1 day before Sunday departure
send_transactional	Operations	Check-in instructions at T-3 days
add / remove_segment_tag	CRM	Tag high_app_sales_potential after second purchase
notify_staff	Operations	Alert manager when a VIP checks in
create_task	Operations	Housekeeping task when early check-in is purchased
start_journey / wait	Orchestration	Multi-step sequences
webhook	Extensibility	Push events to a PMS or external tool

A single WHEN can fan out into mixed THENs — one trigger, marketing and operational consequences: *when deal_purchased (early check-in) → notify housekeeping + tag guest + schedule day-of-arrival upsell.* That composition is where the platform stops being a set of modules and becomes one system.

13 Natural language as the authoring layer

One parser for marketing and operations — plus journeys and proactive Vela

NL → RULE COMPILATION

User: Three days before arrival, if the reservation has children and the guest has email consent, send the early check-in campaign with the family restaurant deal.

Agent: WHEN days_before_arrival(3) · IF children > 0 ∧ consent(email, marketing) [structural, auto-injected] · THEN send_campaign_message(camp_early_family)

Result: Dry run: would have fired 14 times last month, reaching 38 guests, blocking 4 on consent. Draft added to your review queue.

- ✓ **NL in, structure out, always both visible.** The sentence is kept as provenance; the compiled rule is what executes and what the operator edits. Round-trip: editing rule chips regenerates a normalized sentence.
- ✓ **One parser, many domains.** The same grammar handles marketing ("win back summer families") and operations ("notify the cleaner when checkout completes"). Vela routes to the right THEN vocabulary from the verb.
- ✓ **Dry-run before commit.** Every authored rule gets the simulator treatment. Natural language is approachable; the preview is what makes it trustworthy.
- ✓ **Approval model inherited.** Engine rules authored by Vela land as drafts in the same review queue as campaigns. One approval surface for everything the agent wants to do.

From single rules to journeys (forward-looking)

JOURNEY = CHAINED RULES

```
WHEN checkin_abandoned
THEN send reminder (email)
     wait 24h
     IF checkin still incomplete ∧ consent(whatsapp)
     THEN send reminder (whatsapp)
          wait until day_of_arrival
          IF still incomplete THEN notify_staff(front desk)
```

Journeys stay inside the same three-word mental model — a journey is just rules chained on engine events — so the UI, the NL authoring and the approval flow all extend without new concepts.

> **Proactive Vela (forward-looking):** With the engine journaling every run, Vela can propose automations from observed patterns: "Guests who ask about parking convert at 14% when offered within 72 hours — shall I automate this?" The proposal is a drafted when/if/then rule in the review queue, with evidence attached. Same contract as today: Vela proposes, the human approves, the engine executes, consent gates everything.

14 Metrics, non-goals & open questions

How we measure success and what we deliberately do not build

Metric	What it proves
Campaign revenue / month per account	The module makes money
App Sales attach rate (campaign-driven vs baseline)	Campaigns are a real distribution channel
Marketing-consent coverage (% of guests)	The capture-at-check-in thesis works
% AI-drafted campaigns approved without edits	Vela's drafts are good enough to trust
Time from brief → approved campaign	The one-sentence promise holds
Consent blocks at fire time (count, trend)	The safety layer works and is visible
Checklist completion (first-run → first campaign)	Activation flow works

Non-goals

- ✓ **No free-form blasting:** there is no "send email to all guests" path — every send is a campaign with audience, consent check and approval.
- ✓ **No black-box ML:** scoring and recommendations are rule-based and explainable; any future model must satisfy the explainability constraint.
- ✓ **No parallel messaging infrastructure:** delivery, scheduling and events belong to the Automation Engine.
- ✓ **No manual guest data entry:** if a field cannot be derived from operations, the CRM does without it.
- ✓ **No autonomous sending:** the approval requirement is a product identity decision, not a temporary safety measure.

Open questions

- ✓ Engine event emission for dynamic segments — CRM pushes, or engine evaluates segment rules natively?
- ✓ Consent classifier placement — working position: engine middleware; needs Automation Engine team sign-off.
- ✓ Should Vela's NL parser be extracted as a platform service for all modules?
- ✓ Channel expansion sequencing — SMS and push are in the consent matrix but not yet campaign channels.
- ✓ Multi-property semantics — per-property campaign scoping and reporting for group accounts.
- ✓ Journey builder UX — extend the Studio's Automation step, or a dedicated canvas?